



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251176
Nama Lengkap	David Alexsi Cahyadi
Minggu ke / Materi	12 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Materi

Dictionary merupakan salah satu struktur data yang tersedia di Python. Berbeda dengan list yang mengharuskan indeksinya berupa bilangan integer, dictionary jauh lebih fleksibel karena indeksinya bisa berupa tipe data apapun, termasuk string. Dictionary terdiri dari pasangan **kunci:nilai** (*key:value pair*), di mana setiap kunci bersifat unik, tidak boleh ada dua kunci yang sama dalam satu dictionary yang sama. Konsep ini mirip seperti kamus pada umumnya, di mana setiap kata (kunci) memiliki satu arti (nilai) yang bersesuaian.

Untuk membuat dictionary kosong, digunakan fungsi **dict()**. Simbol kurung kurawal **{}** digunakan untuk merepresentasikan dictionary kosong, sedangkan untuk menambahkan item ke dalamnya digunakan tanda kurung kotak **[]**. Sebagai contoh sederhana, berikut cara membuat dictionary yang memetakan kata dalam bahasa Inggris ke bahasa Spanyol:

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp)
{'one': 'uno', 'three': 'tres', 'two': 'dos'}
```

Perlu diperhatikan bahwa urutan pasangan kunci-nilai pada urutan pasangan kunci-nilai pada dictionary dapat dipertahankan sesuai urutan saat dimasukkan. Hal ini tidak menjadi masalah, karena untuk mengakses nilai dalam dictionary tidak menggunakan indeks integer melainkan menggunakan kunci yang bersesuaian. Apabila kunci yang digunakan tidak ditemukan di dalam dictionary, Python akan mengeluarkan pesan error **KeyError**.

Selain itu, dictionary memiliki beberapa fungsi dan operator bawaan yang berguna. Fungsi **len()** digunakan untuk mengetahui jumlah pasangan kunci-nilai. Operator **in** digunakan untuk mengecek apakah suatu kunci ada di dalam dictionary, dan akan mengembalikan nilai **True** atau **False**. Sedangkan untuk mengecek apakah suatu nilai ada di dalam dictionary, dapat digunakan method **.values()** yang terlebih dahulu dikonversi menjadi list.

Dari sisi performa, dictionary menggunakan algoritma **Hash Table** yang sangat efisien. Berbeda dengan list yang menggunakan pencarian linear sehingga semakin panjang list maka semakin lama waktu pencariannya, operator **in** pada dictionary membutuhkan waktu yang relatif konstan tanpa memandang seberapa banyak item yang ada di dalamnya.

Dictionary sebagai Set Penghitung (Counters)

Salah satu kegunaan dictionary yang paling umum adalah sebagai penghitung atau histogram, yaitu menghitung frekuensi kemunculan suatu elemen. Misalnya, jika diberikan sebuah string dan kita diminta menghitung berapa kali setiap huruf muncul, dictionary menjadi pilihan yang paling praktis dibandingkan cara lain.

Cara kerjanya sederhana misalnya, setiap karakter dijadikan kunci, dan jumlah kemunculannya dijadikan nilai. Setiap kali karakter baru ditemukan, dictionary membuat entri baru dengan nilai awal 1. Jika karakter tersebut sudah ada, nilainya cukup ditambah 1.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)
```

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Cara di atas dapat disederhanakan lebih lanjut menggunakan method **.get()** yang dimiliki dictionary. Method ini menerima dua argument, yaitu kunci yang dicari dan nilai default jika kunci tersebut belum ada. Dengan memanfaatkan **.get()**, blok **if-else** yang terdiri dari empat baris dapat dipadatkan menjadi hanya satu baris saja.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     d[c] = d.get(c,0) + 1
5 print(d)
```

Penggunaan **.get()** seperti ini sudah menjadi idiom yang sangat umum dalam pemrograman Python, karena lebih ringkas namun menghasilkan output yang identik.

Dictionary dan File

Dictionary juga sangat berguna untuk mengolah data yang berasal dari file teks. Salah satu penggunaan umumnya adalah menghitung frekuensi kemunculan setiap kata dalam sebuah file. Sebagai contoh, digunakan file **romeo.txt** yang berisi penggalan teks dari drama Romeo dan Juliet (versi tanpa tanda baca).

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Program yang dibuat akan membaca file baris per baris, memecah setiap baris menjadi daftar kata menggunakan **.split()**, lalu menghitung setiap kata menggunakan dictionary. Pola perulangan luar untuk membaca tiap baris dan perulangan dalam untuk mengiterasi setiap kata di baris tersebut.

```

1 fname = input('Enter the file name: ')
2 try:
3     fhand = open(fname)
4 except:
5     print('File cannot be opened:', fname)
6     exit()
7
8 counts = dict()
9 for line in fhand:
10     words = line.split()
11     for word in words:
12         if word not in counts:
13             counts[word] = 1
14         else:
15             counts[word] += 1
16
17 print(counts)

```

Pada kode tersebut juga digunakan penulisan singkat **counts[word] += 1** yang merupakan bentuk ringkas dari **counts[word] = counts[word] + 1**. Pola penulisan serupa juga berlaku untuk operasi aritmatika lain seperti **-=**, ***=**, dan **/=**.

Looping dan Dictionary

Ketika digunakan dalam perulangan **for**, dictionary secara otomatis akan mengiterasi setiap kunci yang ada di dalamnya. Dari kunci tersebut, nilai yang bersesuaian dapat diakses menggunakan operator indeks **[]**.

```

1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     print(key, counts[key])

```

jan 100
 chuck 1
 annie 42

Seperti yang terlihat pada output di atas, kunci tidak ditampilkan dalam urutan tertentu. Namun jika diperlukan urutan tertentu, misalnya urutan alfabet hal ini bisa dilakukan dengan cara mengambil semua kunci menggunakan method **.keys()**, mengonversinya menjadi list, lalu mengurutkannya dengan **.sort()**, dan baru kemudian melakukan perulangan melalui list yang sudah terurut tersebut.

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 lst = list(counts.keys())
3 print(lst)
4 lst.sort()
5 for key in lst:
6     print(key, counts[key])

['jan', 'chuck', 'annie']
annie 42
chuck 1
jan 100
```

Selain itu, perulangan pada dictionary juga dapat dikombinasikan dengan kondisi **if** untuk menyaring data. Misalnya, jika hanya ingin mencetak entri yang nilainya di atas 10, cukup tambahkan pengecekan **if counts[key] > 10** di dalam perulangan.

Advance Text Parsing

Pada bagian sebelumnya, file yang digunakan sudah bersih dari tanda baca. Namun dalam kondisi nyata, teks biasanya mengandung tanda baca seperti koma, titik, tanda seru, dan sebagainya. Hal ini menimbulkan masalah karena Python akan menganggap kata seperti **“soft”** dan **“soft!”** sebagai dua kata yang berbeda. Begitu pula, dengan perbedaan huruf kapital seperti **“Who”** dan **“who”** akan dihitung secara terpisah.

Untuk mengatasi hal ini, Python menyediakan beberapa method string yang bisa dikombinasikan:

- **.lower()**, mengubah seluruh teks menjadi huruf kecil agar tidak ada perbedaan antara **“Who”** dan **“who”**.
- **.rstrip()**, menghapus karakter whitespace (spasi kosong) di akhir baris.
- **.translate()** dan **str.maketrans()**, digunakan untuk menghapus semua karakter tanda baca dari teks.

Daftar karakter yang dianggap sebagai tanda baca oleh Python dapat diperoleh dari modul **string** melalui **string.punctuation**, yang menghasilkan karakter-karakter seperti `'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'`.

```
1  import string
2
3  fname = input('Enter the file name: ')
4  try:
5      fhand = open(fname)
6  except:
7      print('File cannot be opened:', fname)
8      exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '', string.punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21
22 print(counts)
```

Enter the file name: romeo-full.txt

```
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

Dengan menggabungkan ketiga method tersebut sebelum memecah baris menjadi kata-kata, program menjadi jauh lebih akurat dalam menghitung frekuensi kata karena perbedaan kapitalisasi maupun tanda baca tidak lagi mempengaruhi hasil perhitungan.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github: https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 11.1

Soal:

Latihan 11.1 Bualah sebuah program untuk mendapatkan nilai key, value, dan item dari sebuah dictionary.

Contoh:

Dictionary : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

Output:

key	value	item
1	10	1
2	20	2
3	30	3
4	40	4
5	50	5
6	60	6

Source Code:

```
d = {}
n = int(input("Jumlah data : "))

for i in range(n):
    k = int(input(f"Key ke-{i+1} : "))
    v = int(input(f"Value ke-{i+1} : "))
    d[k] = v

print(f"{'key':<8}{'value':<12}{'item'}")
```



```
for i,(k,v) in enumerate(d.items(), start=1):  
    print(f"{k:<8}{v:<12}{i}")
```

Output:

```
PS D:\PrAlpro\Pertemuan 12> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 12/soal01.py"  
Jumlah data : 6  
Key ke-1 : 1  
Value ke-1 : 10  
Key ke-2 : 2  
Value ke-2 : 20  
Key ke-3 : 3  
Value ke-3 : 30  
Key ke-4 : 4  
Value ke-4 : 40  
Key ke-5 : 5  
Value ke-5 : 50  
Key ke-6 : 6  
Value ke-6 : 60  
key      value      item  
1         10         1  
2         20         2  
3         30         3  
4         40         4  
5         50         5  
6         60         6
```

Penjelasan:

Program dimulai dengan membuat sebuah dictionary kosong bernama **d** yang digunakan untuk menyimpan pasangan key dan value. Setelah itu, program meminta pengguna memasukkan jumlah data yang ingin dimasukkan ke dalam dictionary melalui variabel **n**. kemudian, program melakukan perulangan sebanyak jumlah data tersebut menggunakan **for**. Pada setiap perulangan, pengguna diminta memasukkan nilai key dan value. Nilai key disimpan pada variabel **k**, sedangkan value disimpan pada variabel **v**. Kedua nilai tersebut kemudian dimasukkan ke dalam dictionary menggunakan perintah **d[k] = v**.

Setelah seluruh data selesai dimasukkan, program menampilkan judul tabel yaitu **key**, **value**, dan **item** menggunakan formatted string agar tampilan menjadi lebih rapi dan sejajar. Selanjutnya, program menggunakan **enumerate(d.items(), start=1)** untuk mengambil nomor urut item beserta pasangan key dan value dari dictionary. Method **items()** digunakan untuk mengambil seluruh pasangan key dan value pada dictionary, sedangkan **enumerate()** digunakan untuk memberikan nomor urut mulai dari 1. Pada setiap perulangan, program

menampilkan key, value, dan nomor item ke layar sehingga menghasilkan output berbentuk tabel seperti pada soal.

Latihan 11.2

Soal:

Latihan 11.2 Bualah sebuah program untuk memetakan dua list mejadi satu dictionary.

Contoh:

Data List

```
Lista = ['red', 'green', 'blue']
```

```
Listb = ['#FF0000', '#008000', '#0000FF']
```

Output

```
{'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}
```

Source Code:

```
print("Data List")
la = eval(input("Lista = "))
lb = eval(input("Listb = "))
hasil = dict(zip(la,lb))
print(hasil)
```

Output:

```
PS D:\PrAlpro\Pertemuan 12> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 12/soal02.py"
Data List
Lista = ['red', 'green', 'blue']
Listb = ['#FF0000', '#008000', '#0000FF']
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

Penjelasan:

Program dimulai dengan menampilkan tulisan “Data List” sebagai penanda bahwa pengguna akan memasukkan data list. Setelah itu, program meminta pengguna memasukkan list pertama dan menyimpannya ke dalam variabel **la** menggunakan **eval(input())**. Kemudian program kembali meminta pengguna memasukkan list kedua dan

menyimpannya ke variabel **lb**. Kedua list tersebut digunakan sebagai key dan value pada dictionary.

Selanjutnya, program menggunakan fungsi **zip(la,lb)** untuk memasangkan setiap elemen dari list pertama dengan elemen pada list kedua berdasarkan urutan indeksinya. Hasil pasangan tersebut kemudian diubah menjadi dictionary menggunakan fungsi **dict()** dan disimpan ke dalam variabel **hasil**. Program menampilkan isi dictionary ke layar menggunakan **print(hasil)** sehingga menghasilkan gabungan dua list menjadi sebuah dictionary sesuai dengan soal.

Latihan 11.3

Soal:

Latihan 11.3 Dengan menggunakan file **mbox-short.txt**, buatlah program yang dapat membaca log email dan sajikan dalam histogram menggunakan dictionary. Kemudian hitung berapa banyak pesan yang masuk dari email dan sajikan dalam bentuk dictionary.

Silakan cek bagian dibawah ini untuk contoh output dari programnya.

```
Masukkan nama file : mbox-short.txt
```

```
{'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3,
'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1,
'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3,
'david.horwitz@uct.ac.za': 4, 'wagnermr@iupui.edu': 1,
'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2,
'ray@media.berkeley.edu': 1}
```

Source Code:

```
nf = input("Masukkan nama file : ")
try:
    buka = open(nf)
except FileNotFoundError:
    print("File tidak bisa dibuka:",nf)
    exit()
jumlah = dict()
```

```

for baris in buka:
    if not baris.startswith('From '):
        continue
    pecah = baris.split()
    alamat = pecah[1]
    jumlah[alamat] = jumlah.get(alamat, 0) + 1
print(jumlah)

```

Output:

```

PS D:\PrAlpro\Pertemuan 12> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 12/soal03.py"
Masukkan nama file : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnerm@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}

```

Penjelasan:

Program dimulai dengan meminta pengguna memasukkan nama file yang akan dibaca. Setelah itu, program mencoba membuka file menggunakan **try-except** untuk mencegah terjadinya error apabila file tidak ditemukan. Jika file gagal dibuka, program akan menampilkan pesan kesalahan lalu menghentikan proses menggunakan **exit()**. Selanjutnya dibuat sebuah dictionary bernama **jumlah** yang digunakan untuk menyimpan jumlah kemunculan setiap alamat email.

Program kemudian membaca isi file baris demi baris menggunakan perulangan **for**. Setiap baris diperiksa apakah diawali dengan kata **"From"**. Jika tidak sesuai, maka baris tersebut dilewati menggunakan **continue**. Jika sesuai, program memisahkan isi baris menjadi beberapa kata menggunakan **split()**, kemudian mengambil alamat email yang berada pada indeks ke-1 dan menyimpannya ke variabel **alamat**.

Setelah alamat email diperoleh, program akan menghitung jumlah kemunculan email tersebut menggunakan dictionary **jumlah**. Method **get()** digunakan untuk mengambil nilai sebelumnya, dan jika email belum ada maka nilainya dianggap 0. Nilai tersebut kemudian

ditambah 1 setiap kali email ditemukan. Setelah seluruh file selesai dibaca, program menampilkan dictionary yang berisi daftar alamat email beserta jumlah pesan dari masing-masing email.

Latihan 11.4

Soal:

Latihan 11.4 Dengan menggunakan file **mbox-short.txt**, buat program untuk mencatat data nama domain pengirim pesan. Hitunglah jumlah pesan yang dikirim masing-masing domain. sajikan dalam bentuk dictionary.

Silakan cek bagian dibawah ini untuk contoh output dari programnya.

Masukkan nama file: mbox-short.txt

```
{'media.berkeley.edu': 4, 'uct.ac.za': 6, 'umich.edu': 7,  
'gmail.com': 1, 'caret.cam.ac.uk': 1, 'iupui.edu': 8}
```

Source Code:

```
nf = input("Masukkan nama file : ")  
try:  
    buka = open(nf)  
except FileNotFoundError:  
    print("File tidak bisa dibuka:",nf)  
    exit()  
jumlah = dict()  
for baris in buka:  
    if not baris.startswith('From '):  
        continue  
    pecah = baris.split()  
    email = pecah[1]  
    domain = email.split('@')[1]  
    jumlah[domain] = jumlah.get(domain, 0) + 1  
print(jumlah)
```

Output:

```
PS D:\PrAlpro\Pertemuan 12> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 12/soal04.py"  
Masukkan nama file : mbox-short.txt  
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}
```

Penjelasan:

Program dimulai dengan meminta pengguna memasukkan nama file yang akan dibaca. Setelah itu, program mencoba membuka file menggunakan **try-except** untuk mencegah terjadinya error apabila file tidak ditemukan. Jika file gagal dibuka, program akan menampilkan pesan kesalahan lalu menghentikan proses menggunakan **exit()**. Selanjutnya dibuat sebuah dictionary bernama **jumlah** yang digunakan untuk menyimpan domain email.

Program kemudian membaca isi file baris demi baris menggunakan perulangan **for**. Setiap baris diperiksa apakah diawali dengan kata **“From ”**. Jika tidak sesuai, maka baris tersebut dilewati menggunakan **continue**. Jika sesuai, program memisahkan isi baris menjadi beberapa kata menggunakan **split()**, kemudian mengambil alamat email yang berada pada indeks ke-1 dan menyimpannya ke variabel **email**.

Setelah alamat email diperoleh, program mengambil nama domain dengan memisahkan email berdasarkan tanda **@** menggunakan **split('@')**. Bagian domain yang berada pada indeks ke-1 disimpan ke variabel **domain**. Program kemudian menghitung jumlah kemunculan setiap domain menggunakan dictionary **jumlah**. Method **get()** digunakan untuk mengambil nilai sebelumnya, dan jika domain belum ada maka nilainya dianggap 0. Nilai tersebut kemudian ditambah 1 setiap kali domain ditemukan. Setelah seluruh file selesai dibaca, program menampilkan dictionary yang berisi daftar domain beserta jumlah pesan yang dikirim dari masing-masing domain.